

P0334r0 : Immutable Persistent Containers

Project: ISO JTC1/SC22/WG21: Programming Language C++
Number: P0334r0
Date: 2016-04-24
Reply-to: balelbach@lbl.gov
Author: Bryce Adelstein Lelbach
Contact: balelbach@lbl.gov
Audience: Library Evolution Working Group (LEWG)
URL: <https://git.io/vr7qi>

1 Motivation

While discussing a more general-purpose `exception_list` (see P0322), my co-authors and I began to explore the broader design space of immutable, persistent containers. These data structures, prevalent in functional programming languages, such as Haskell or Lisp, are useful in parallel programming contexts as they are inherently safe to access concurrently (because they cannot be modified).

2 Design

The basic properties of an immutable container are the following:

- Cheap to "copy", "concatenate" and "split". Typically a "copy" operation is merely a copy of a reference-counted pointer (in languages like Haskell, garbage collection is used).
- When "modifying" the container, older versions of the container stay around (e.g. persistent). No operation is truly mutating.

From these properties, it is straightforward to see where immutable containers might be useful:

- When we want containers provide efficient concatenation and splitting operations - for example, [the rope<> string class from SGI's STL library](#). Such a data structure can very useful when implementing a text editor or parser.
- When we want containers which are safe to use in parallel contexts - for example, `exception_list` could be an immutable list. Immutable data structures do not rely on synchronization primitives such as mutexes or atomics. Concurrent access is safe because all concurrent accesses are reads. However, using immutable data structures for concurrency places restricts the form of the parallel algorithms operating on the immutable data structures. Other concurrent data structures (such as a theoretical `atomic_list<>`) would arguably be more flexible, at the cost of being forced to use synchronization primitives to protect the data structure. However, immutable data structures are a powerful tool for parallel programmers to have in their pocket, and we should consider adding them to the language to support this use case.

In Haskell, immutable sequences are implemented using [2-3 finger trees](#). These trees allow for constant time access to and insertion at the ends of the sequence and logarithmic time splitting and concatenation. A non-lazy, reference-counted implementation of 2-3 finger trees would be one possible way of efficiently implementing `rope<>` and `exception_list` in C++. Other algorithms allow for constant time concatenation at the cost of linear time splitting.

As far as interface design, for `exception_list` we decided to provide the traditional `Container` interfaces for accessing elements and properties of the container. We created a set of constructors which allow for splitting and concatenation. Alternatively, a set of free function interfaces serve as the interface for this functionality. This design could be extended from `exception_list` (a sequence of `exception_ptr`s) to `immutable_list<T>` (a sequence of `T`s) and other possible immutable types (such as a `rope<>`-like type).

A draft specification for an `immutable_list<>` is provided below. It is intended to be implemented with a referenced-counted tree structure such as a 2-3 finger tree; complexity guarantees reflect this. The move constructor and move assignment operator have been deleted, as they are modifying operations and `immutable_list<>` is an immutable type.

3 Specification

```
namespace std {
```

```

template <class T, class Allocator = std::allocator<T> >
class immutable_list
{
public:
    typedef T value_type;
    typedef Allocator allocator_type;
    typedef /** unspecified */ size_type;
    typedef /** unspecified */ difference_type;
    typedef value_type& reference;
    typedef value_type const& const_reference;
    typedef typename allocator_traits<Allocator>::pointer pointer;
    typedef typename allocator_traits<Allocator>::const_pointer const_pointer;
    typedef /** unspecified */ iterator;
    typedef /** unspecified */ const_iterator;

    //////////////////////////////////////
    // CONSTRUCTORS

    constexpr immutable_list() noexcept = default;

    immutable_list(const immutable_list& other);
    immutable_list& operator=(const immutable_list& other);

    immutable_list(immutable_list&&) = delete;
    immutable_list& operator=(immutable_list&&) = delete;

    // "push_back" and "emplace_back" constructors
    immutable_list(T const& v);
    immutable_list(T&& v);
    immutable_list(const immutable_list& other, T const& v);
    immutable_list(const immutable_list& other, T&& v);

    // iterator-pair "insert" constructors
    immutable_list(iterator first, iterator last);
    immutable_list(const immutable_list& other,
                    iterator first, iterator last);
    template <class InputIterator>
    immutable_list(InputIterator first, InputIterator last);
    template <class InputIterator>
    immutable_list(const immutable_list& other,
                    InputIterator first, InputIterator last);

    // initializer-list "insert" constructors
    immutable_list(initializer_list<T> list);
    immutable_list(const immutable_list& other,
                    initializer_list<T> list);

    // "splice" constructor
    immutable_list(const immutable_list& other0,
                    const immutable_list& other1) noexcept;

    //////////////////////////////////////
    // QUERY INTERFACE

    size_type size() const noexcept;

    iterator begin() const noexcept;
    iterator cbegin() const noexcept;

    iterator end() const noexcept;
    iterator cend() const noexcept;
};
}

```

The class `immutable_list<T>` owns a sequence of `T` objects.

The type `immutable_list<T>::iterator` shall fulfill the requirements of `ForwardIterator`.

The type `immutable_list<T>::size_type` shall be an unsigned integral type large enough to represent the size of the sequence.

The type `immutable_list<T>::difference_type` shall be an unsigned integral type large enough to represent distances between iterators to the sequence.

`constexpr immutable_list() noexcept = default;`

Effect: Construct an empty `immutable_list`.

```
immutable_list(const immutable_list& other);
```

Effect: Construct a new `immutable_list` which is a copy of `other`.

Complexity: Linear time in the size of `other`.

Complexity: Constant time.

```
immutable_list& operator=(const immutable_list& other);
```

Effect: Copy the contents of `other` into this `immutable_list`.

Complexity: Linear time in the size of `other`.

Complexity: Constant time.

```
immutable_list(T const& v);
```

Effect: Construct a new `immutable_list` which contains a single element which is a copy of `v`.

Complexity: Constant time.

```
immutable_list(T&& v);
```

Effect: Construct a new `immutable_list` which contains a single element which has been moved from `v`.

Complexity: Constant time.

```
immutable_list(const immutable_list& other, T const& v);
```

Effect: Construct a new `immutable_list` which is a copy of `other`, and append a new element which is a copy of `v` to the end of the owned sequence.

Complexity: Linear in the size of `other` + 1.

Complexity: Constant time.

```
immutable_list(const immutable_list& other, T&& v);
```

Effect: Construct a new `immutable_list` which is a copy of `other`, and append a new element which is moved from `v` to the end of the owned sequence.

Complexity: Linear in the size of `other` + 1.

Complexity: Constant time.

```
immutable_list(iterator first, iterator last);
```

Effect: Construct a new `immutable_list` which contains `distance(first, last)` elements from the range `[first, last)`.

Complexity: Logarithmic in `distance(first, last)`.

```
immutable_list(const immutable_list& other, iterator first, iterator last);
```

Effect: Construct a new `immutable_list` which is a copy of `other`, and append the range `[first, last)` to the end of the owned sequence.

Complexity: Logarithmic in `min(other.size(), distance(first, last))`.

```
template <class InputIterator>
```

```
immutable_list(InputIterator first, InputIterator last);
```

Effect: Construct a new `immutable_list` which contains `distance(first, last)` elements from the range `[first, last)`.

Complexity: Linear in `distance(first, last)`.

Remarks: This constructor shall not participate in overload resolution if `is_convertible_v<typename InputIterator::value_type, T> == false`.

```
template <class InputIterator>
```

```
immutable_list(const immutable_list& other, InputIterator first, InputIterator last);
```

Effect: Construct a new `immutable_list` which is a copy of `other`, and append the range `[first, last)` to the end of the

owned sequence.

Complexity: Linear in `distance(first, last)`.

Remarks: This constructor shall not participate in overload resolution if `is_convertible_v<typename InputIterator::value_type, T> == false`.

```
immutable_list(initializer_list<T> list);
```

Effect: Construct a new `immutable_list` which contains `list.size()` elements from `list`.

Complexity: Linear in the size of `list`.

```
immutable_list(const immutable_list& other, initializer_list<T> list);
```

Effect: Construct a new `immutable_list` which is a copy of `other`, and append `list` to the end of the owned sequence.

Complexity: Linear in the size of `list`.

```
immutable_list(const immutable_list& other0, const immutable_list& other1);
```

Effect: Construct a new `immutable_list` which contains all the elements of `other0` followed by all the elements of `other1`.

Complexity: Logarithmic in the `min(other0.size(), other1.size())`.

```
size_type size() const noexcept;
```

Returns: The number of `T` objects contained within the `immutable_list`.

Complexity: Constant time.

```
iterator begin() const noexcept;
```

```
iterator cbegin() const noexcept;
```

Returns: An iterator referring to the first `T` object contained within the `immutable_list`.

```
iterator end() const noexcept;
```

```
iterator cend() const noexcept;
```

Returns: An iterator that is past the end of the owned sequence.